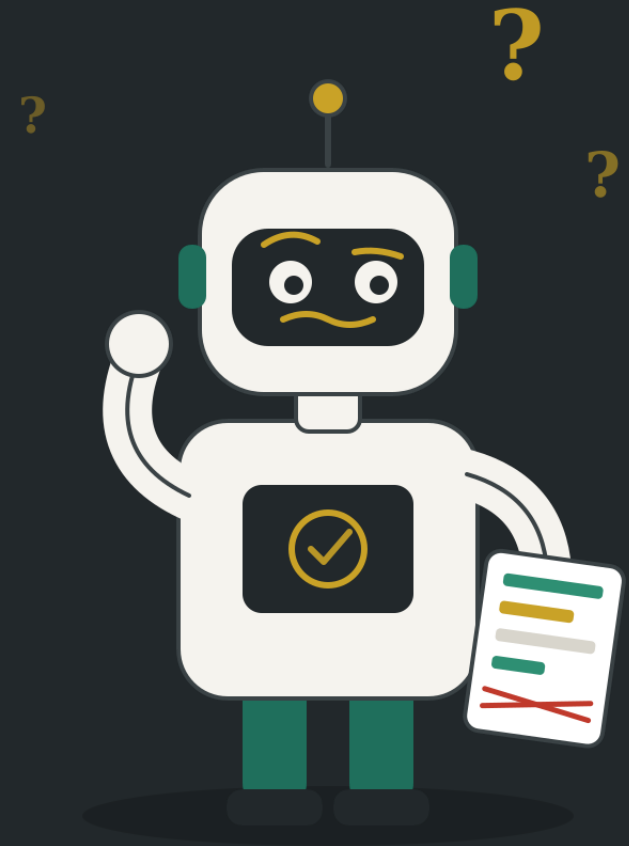


POWER QUERY CASE STUDY

Duplicates. Blanks. No Structure. Can Power Query Fix This?

A step-by-step transformation of a raw HR export into a clean, analysis-ready dataset - built entirely in Power Query.





One Flawed HR Export, Seven Fixes to Make

The HR team's source file tracks six fields per employee - Full Name, Email, Gender, Salary, Department and Hire Date. Before it can support any analysis, the raw export needs deliberate, structured cleanup.

- 1 Remove rows and columns irrelevant to analysis
- 2 Promote the first row to column headers
- 3 Remove duplicate and blank records
- 4 Replace empty Email cells with "No Email"
- 5 Split Full Name into First and Last Name
- 6 Sort the dataset by Hire Date, ascending
- 7 Insert an Employee ID column and move it to the front



Before Fixing Anything, the Data Was Audited

The source file was imported directly from a CSV export, with no header row promoted and no formatting applied. A column-by-column quality check surfaced exactly what needed attention.

Two fully empty columns

Column7 and Column8 carried 100% empty values - leftover artefacts with no analytical value.

Inconsistent completeness

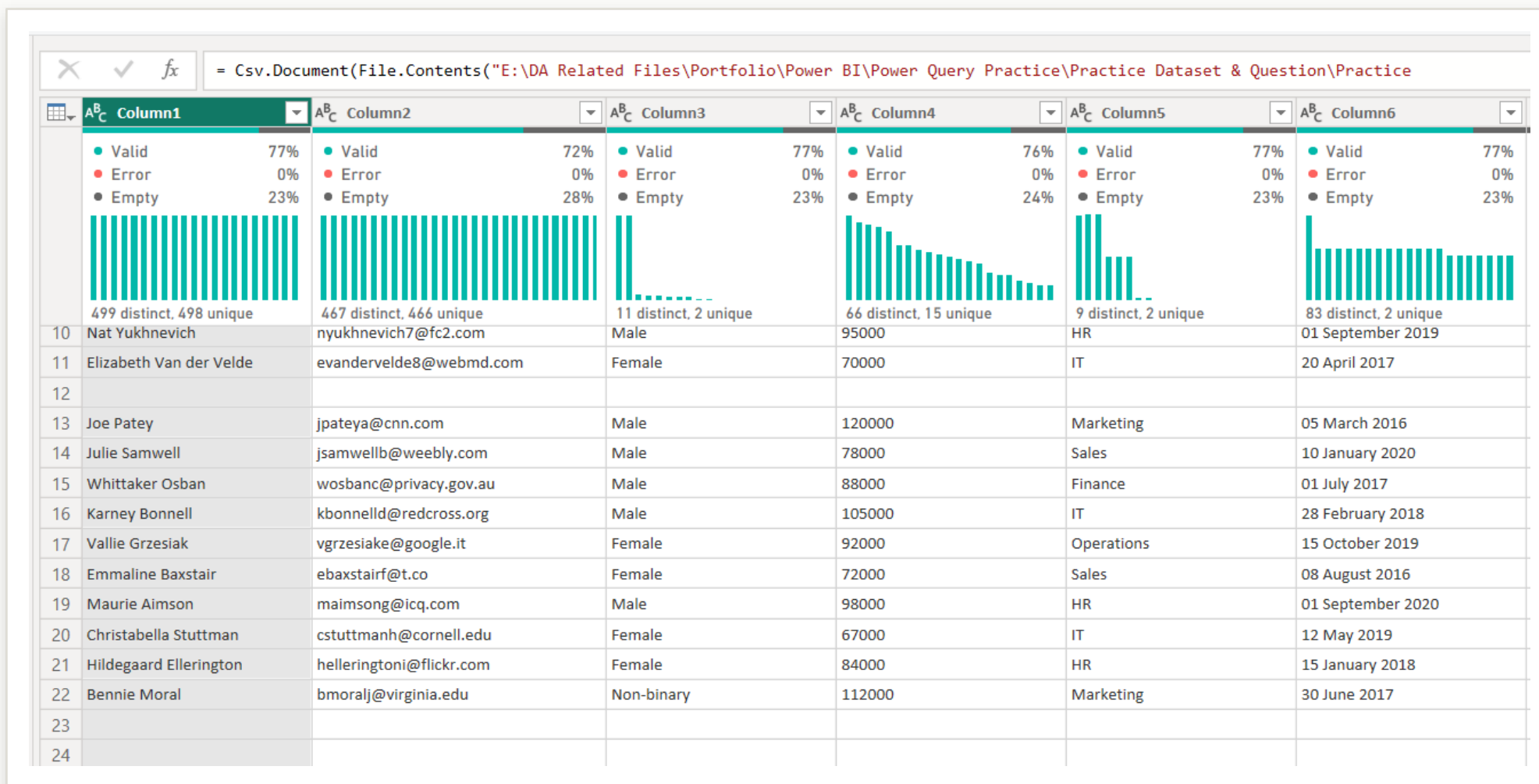
Core fields ranged from 72% to 77% valid, with the rest split between blanks and duplicates.

No usable header row

Fields appeared as generic Column1-Column8 rather than named, typed attributes.

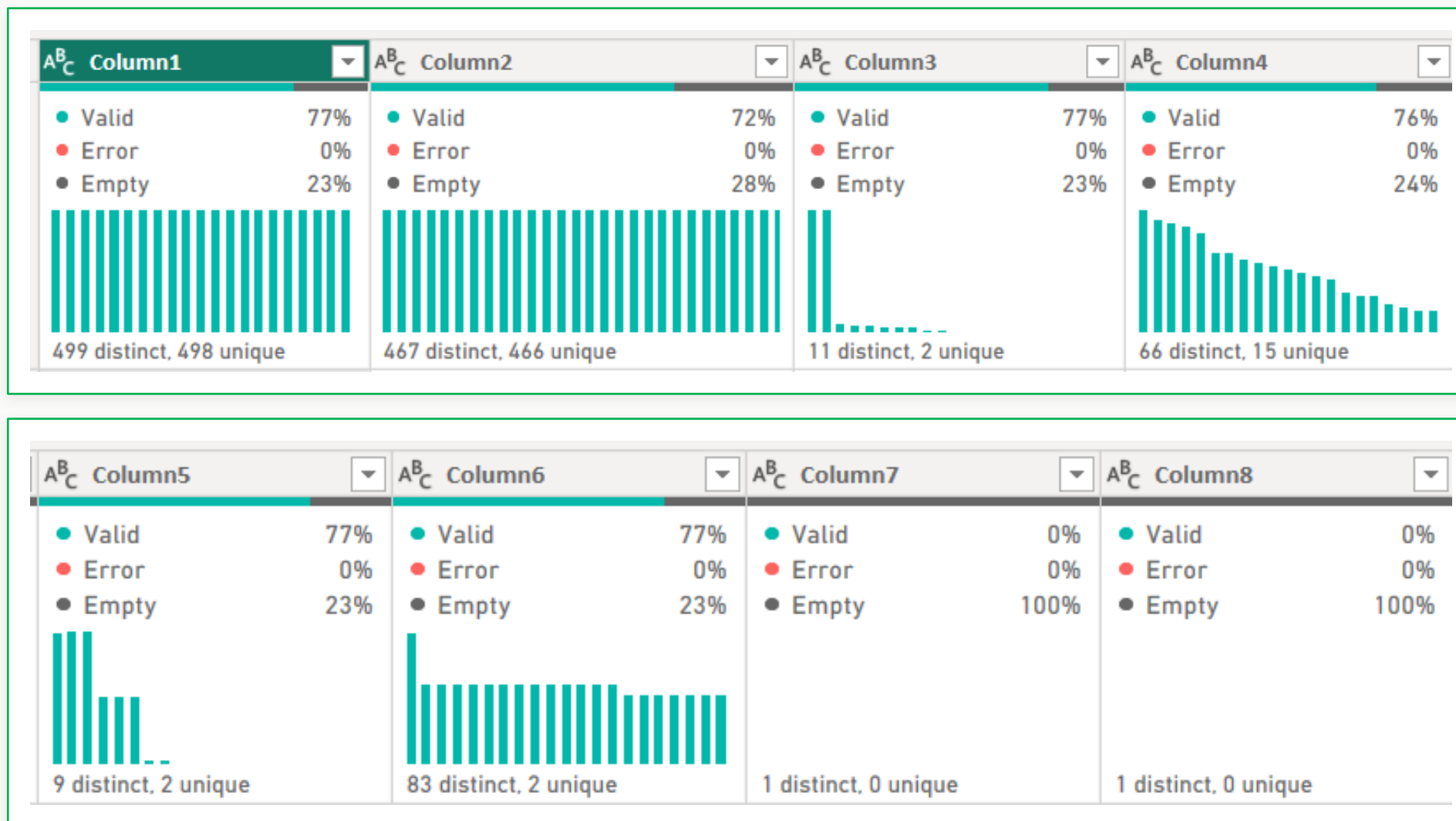
STEP 01 · DIAGNOSE

The Raw Import, Straight From Source



Csv.Document(File.Contents(...)) - unfiltered source connection inside Power Query.

Column-Level Quality Check, Pre-Cleanup





Clearing the Structural Noise

- Removed the top row that held no usable data before the true header row
- Promoted the first data row to serve as proper column headers
- Removed Column7 and Column8 entirely - both were 100% empty
- Removed blank rows so every remaining record carries real employee data
- Removed duplicate entries to keep the dataset free of repeated records

2

empty columns removed

496

clean records retained



Giving Every Blank Cell a Clear Meaning

Blank cells are ambiguous - they don't tell you whether data is missing, unknown, or simply not collected. Two targeted Replace Values steps removed that ambiguity for the fields that needed it most.



Missing Email

Blank → “No Email”

Keeps every row analysis-ready without inventing an address.



Missing Salary

null → 0

Preserves numeric integrity so Salary stays aggregatable.

Replace Values - Email Column

×

Replace Values

Replace one value with another in the selected columns.

Value To Find

Replace With

▸ Advanced options

OK

Cancel

Value to find left blank, Replace With set to "No Email" - applied to the Email column only.

STEP 03 • STANDARDIZE

Replace Values - Salary Column

×

Replace Values

Replace one value with another in the selected columns.

Value To Find

Replace With

OK

Cancel

null values in Salary were replaced with 0, keeping the field numeric and chart-ready.



One Name Field Isn't Enough for Analysis

A single Full Name column blocks first-name and last-name level reporting. Splitting it by delimiter unlocks that granularity - the level of detail a dataset can be broken down to.

- Split by the Space delimiter, at the left-most occurrence
- Produced two clean fields: Employee First Name and Employee Last Name
- Both columns renamed and type-checked immediately after the split

Split Column by Delimiter - Full Name

×

Split Column by Delimiter

Specify the delimiter used to split the text column.

Select or enter delimiter

Space

Split at

☒ Left-most delimiter

☐ Right-most delimiter

☐ Each occurrence of the delimiter

▸ Advanced options

Quote Character

"

☐ Split using special characters

Insert special character

OK

Cancel

Delimiter: Space · Split at: Left-most delimiter - separates first name from the remainder.



Putting Tenure in Order

With names, emails and salaries standardized, the dataset was sorted by Hire Date in ascending order - surfacing the longest-tenured employees first and giving any downstream timeline analysis a reliable starting point.

Sort by Hire Date

A → Z

Ascending, oldest hire date first

80 distinct hire dates spanning the full workforce



Engineering a Unique Employee ID

The dataset had no unique identifier, so one was built in two stages: a sequential index, then a formatted ID derived from it.

1

Add Index Column

Starting index 1, increment 1 - one sequential number per row.

2

Add Custom Column

= "EM" & Text.PadStart(Text.From([Index]), 2, "0") - builds an EM-prefixed, zero-padded ID.

Add Index Column

×

Add Index Column

Add an index column with a specified starting index and increment.

Starting Index

1

Increment

1|

OK

Cancel

Starting Index: 1 · Increment: 1 - the foundation for the Employee ID formula.

Custom Column - Employee ID Formula

Custom Column

Add a column that is computed from the other columns.

New column name

Custom column formula ⓘ

```
= "EM" & Text.PadStart(Text.From([Index]), 2, "0")
```

[Learn about Power Query formulas](#)

✓ No syntax errors have been detected.

OK Cancel

Available columns

- Employee First Name
- Employee Last Name
- Email
- Gender
- Salary
- Department
- Hire Date

<< Insert

"EM" + zero-padded index → consistent, sortable IDs such as EM01, EM02 ...

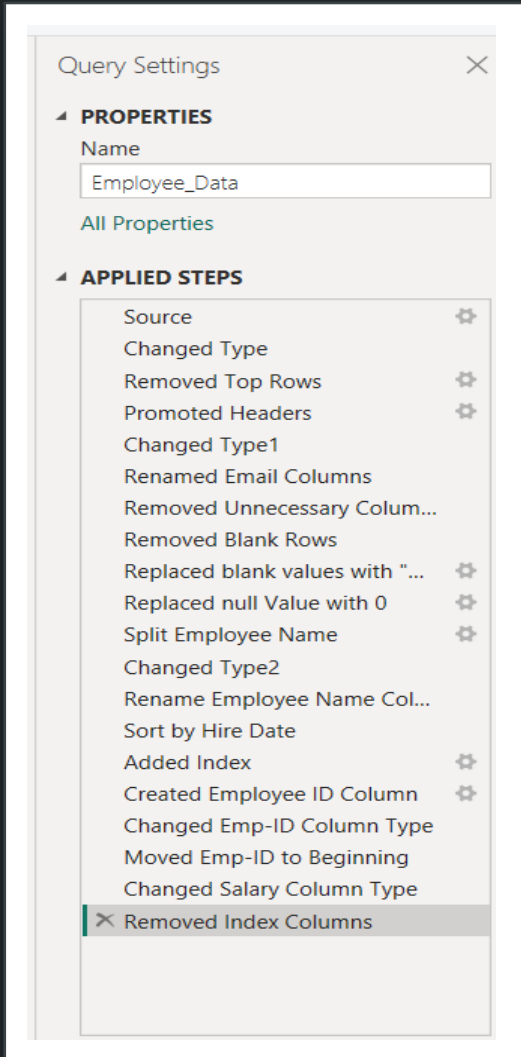


Closing Out the Transformation

- Converted Employee ID to the correct data type immediately after creation
- Moved Employee ID to the very first position in the dataset
- Confirmed Salary's column type after all replacements were applied
- Removed the helper Index column - its job was done once the ID existed

THE COMPLETE PIPELINE

Every Applied Step, in Order



● Import & Structure

Source, type detection, header promotion, column removal

● Cleaning

Blank row removal, Email and Salary replacements

● Transformation

Name split, type fixes, sort by Hire Date

● Enrichment

Index, Employee ID formula, ID repositioning



Proving the Dataset Is Analysis-Ready

The same column-quality check used to diagnose the raw file was run again after every step. Every field now returns 100% valid, 0% error and 0% empty - the dataset is ready to move into Power BI.

496

unique employee records

9

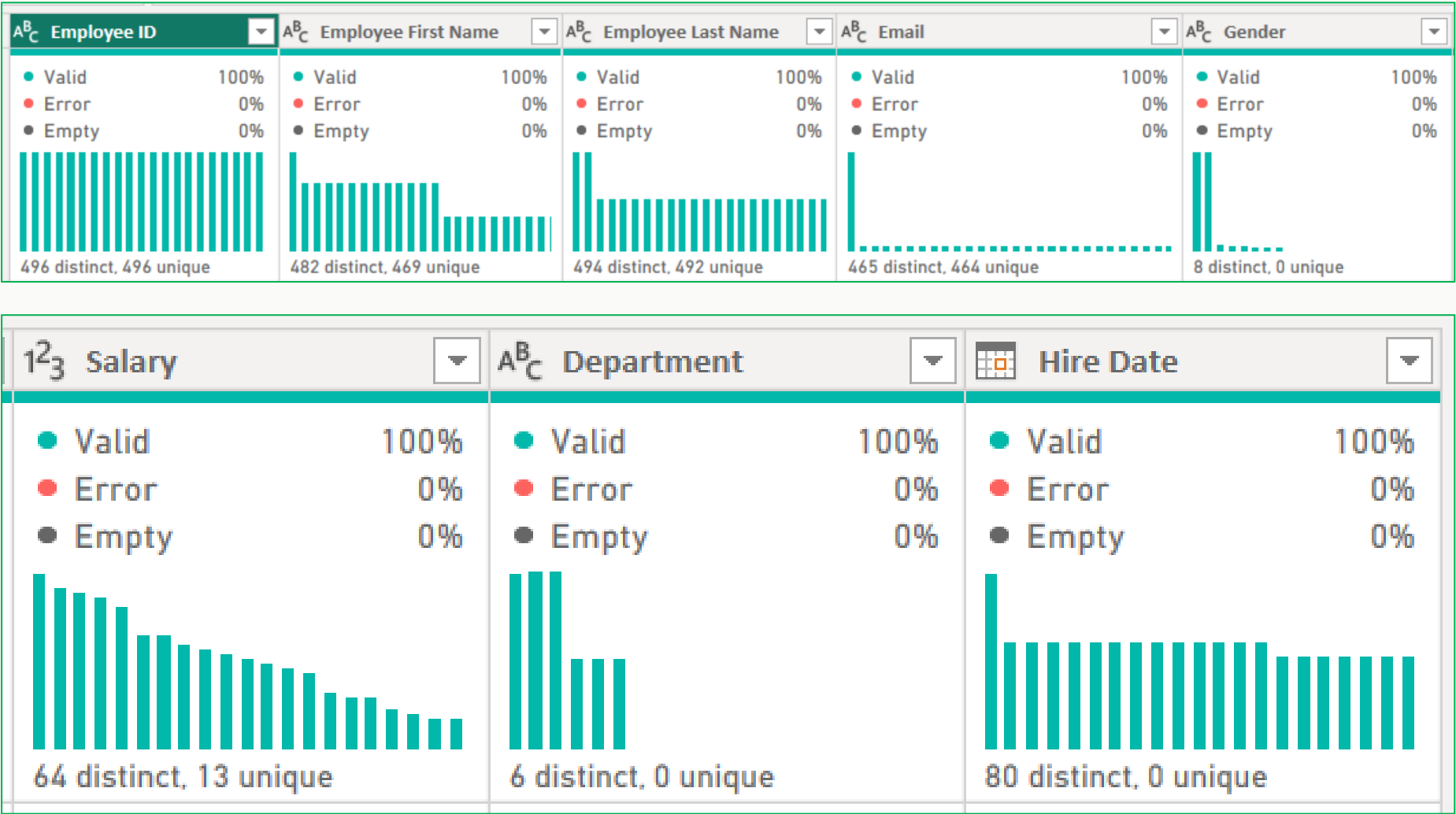
clean, typed columns

100%

valid across every field

STEP 08 · VALIDATE

Column Quality Check, Post-Cleanup



Every column - Employee ID through Hire Date - now reads 100% Valid.